

Atividade Prática - Representação do Mundo

Disciplina: Inteligência Artificial para Jogos I

Aluno: CAUA DANTAS DE SOUZA

Link da solução: <https://github.com/Cauadsouza/Representacao-Mundo-Simples>

Descrição da solução

A solução desenvolvida consiste em um jogo 2D simples no qual o cenário é dividido em nove áreas organizadas em uma malha 3x3. O jogador pode se movimentar pelo mundo e encontrar inimigos e itens de vida e munição. Os NPCs utilizam uma automatização simples de perseguição, calculando a direção até o jogador e se deslocando em linha reta.

A divisão em áreas foi usada principalmente para limitar quais inimigos precisam ser processados. Cada área armazena os NPCs e itens que pertencem àquela região. A área onde o jogador se encontra fica ativa e, quando ele se aproxima de uma borda, a área vizinha correspondente também é ativada. Próximo de um canto, a área diagonal pode ser ativada, mantendo no máximo quatro áreas ativas ao mesmo tempo. Apenas os NPCs dessas áreas executam sua atualização.

Organização do mundo e viewport

As posições do jogador, NPCs e itens são armazenadas usando coordenadas do mundo. A câmera é tratada separadamente e apenas determina qual parte do cenário aparece na janela. Dessa forma, o viewport não depende da divisão 3x3 utilizada pela lógica do jogo.

Quando um NPC atravessa a borda de uma área, sua nova posição é usada para identificar em qual célula da malha ele passou a estar. A mudança é aplicada depois da atualização dos NPCs daquele frame, evitando alterar a coleção enquanto ela está sendo percorrida.

Interações e testes

O jogador começa com vida e uma pequena quantidade de munição. Itens de vida recuperam saúde, enquanto itens de munição permitem usar um ataque que causa dano nos inimigos próximos. O jogador perde quando sua vida chega a zero e vence se permanecer vivo até o fim do tempo definido.

Para observar melhor o funcionamento da divisão do mundo, foi incluído um modo de debug que mostra a área atual, o número de áreas ativas, a quantidade total de NPCs e quantos deles foram atualizados naquele frame. Também foram criados três cenários de teste com 10, 50 e 200 inimigos por área.

Execução

O projeto foi escrito em C++17 e utiliza raylib. A compilação pode ser feita com CMake:

```
cmake -S . -B build
cmake --build build -j
./build/representacao_mundo
```

O código-fonte está disponível em

<https://github.com/Cauadsouza/Representacao-Mundo-Simples>.